The Whole Pipeline, Explained Gently

From colouring a graph to lighting up atoms — and back again

A plain-language companion to the paper

June 3, 2026

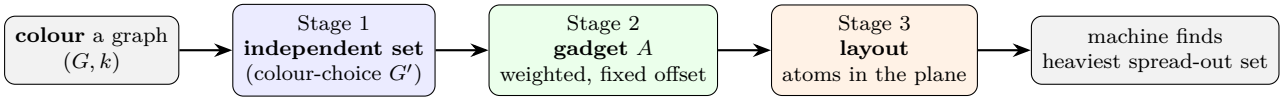
Abstract

This note explains, in the simplest terms I can manage, every step that turns a graph-colouring question into something a neutral-atom machine can solve, and how we read the answer back out. We follow *one tiny example* the whole way through. No proofs, no heavy notation — just what each step does and why. The order is the order the machine sees it: colouring → independent set → gadget → the seven layout heuristics → decomposition (for big graphs) → recovering the colours.

Contents

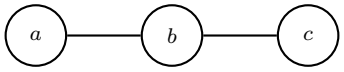
1 The journey in one breath

We want to **colour the vertices of a graph** with k colours so that no edge joins two same-coloured vertices. The hardware cannot do this directly. It can only do *one* thing: take a bunch of atoms placed in the plane and find the heaviest set of atoms with no two of them sitting close together. So we translate, in three exact steps, and one geometric step:



In plain words. Stages 1 and 2 are *exact* translations — they never change the answer, only its form. Stage 3 is *geometry* — it decides whether the machine can physically fit the problem, but it can never give a wrong colour. At the end we run the arrows backwards to read the colours off.

Our running example. The smallest graph that shows everything: a path of three vertices, $a - b - c$, and we ask for a **2-colouring** ($k = 2$, colours 1 and 2).



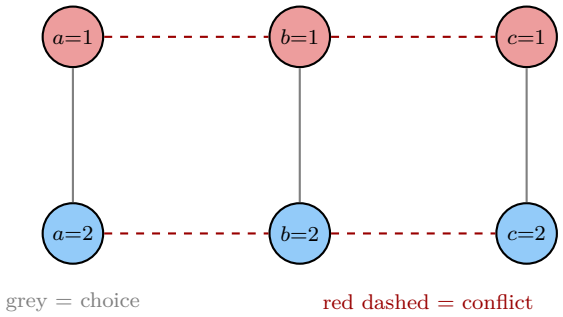
An obvious answer is $a = 1,\ b = 2,\ c = 1$. Let us watch the machinery rediscover it.

2 Stage 1: colouring → independent set (the colour-choice graph)

The trick. Make one little "light switch" for every (vertex, colour) pair. The switch (v, i) means *"vertex v takes colour i."* For our example with 2 colours that is $3 \times 2 = 6$ switches. Now we wire them with two kinds of *forbidding* edges:

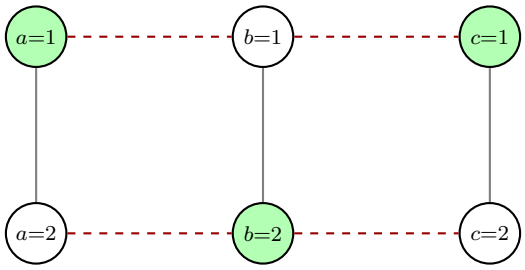
- **Choice edges** (one column per vertex): the two switches of the same vertex are joined, because a vertex may take *at most one* colour. You cannot turn on both $(a, 1)$ and $(a, 2)$.
- **Conflict edges** (between neighbours): if $a - b$ is an edge of the graph, join $(a, 1)$ to $(b, 1)$ and $(a, 2)$ to $(b, 2)$, because neighbours may *not* share a colour. You cannot turn on $(a, 1)$ and $(b, 1)$ together.

This wired-up graph is the **colour-choice graph** G'. A set of switches with *no forbidding edge between any two of them* is exactly an *independent set*.



In plain words. A *proper k-colouring* of the original graph is the *same thing* as an independent set that switches on *exactly one switch per vertex* (that is, a set of size n). "Colour the graph" has become "find a big independent set."

Reading our example. Turn on (a=1), (b=2), (c=1):



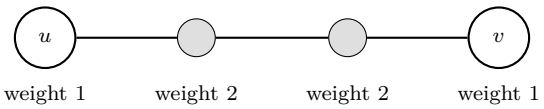
No two green switches are joined: (a=1) and (b=2) are in different rows (no conflict), a and c are not neighbours, and we used one switch per column. Three switches on = three vertices coloured = a valid 2-colouring. If the graph had *needed* more colours, no size-3 independent set would exist, and that is how the machine would say "impossible."

(In the code this is `UDGColor.Stage1.colorChoiceGraph`; the size of the biggest independent set is checked against a brute-force oracle on all small graphs.)

3 Stage 2: independent set → gadget (the weighted graph A)

The machine does not solve plain "independent set"; it solves **maximum-*weight* independent set** (MWIS) on a graph where every edge must come from atoms being physically close. Stage 2 rebuilds G' into a weighted graph A that the machine likes, *without changing which independent sets win.*

The gadget, on one edge. Replace each forbidding edge $\{u, v\}$ of G' by a little **chain of four**: keep the two original switches (now called *logical atoms*, weight 1) and insert two new helper atoms between them (*gadget atoms*, weight 2):



Why four in a chain? Look at how much weight this little chain can contribute to a spread-out (independent) set:

- Take *one* helper atom: weight 2. (The two helpers are joined, so you cannot take both.)
- You may also take *one* of the end switches alongside a helper, as long as they are not adjacent — so a single chain happily gives you weight 2 plus maybe an endpoint.
- But if you greedily grab *both* end switches u *and* v, the chain forces you to drop *both* helpers, and you lose that weight-4 of helpers to gain weight-2 of endpoints. Bad trade.

So the chain quietly *punishes* taking both endpoints — which is exactly the old forbidding edge "you may not take both u and v," now enforced by weights instead of a hard rule.

In plain words. Every edge-chain pays the machine a fixed bonus no matter what, *unless* you try to switch on both of its endpoints. Add up the bonuses and you get a fixed number $C = 2 \times$ (number of edges of G'). The heaviest set of A is then the best independent set of G' *plus* that constant:

$$\mathrm{MWIS}(A) \;=\; \alpha(G') \;+\; C.$$

The constant carries no information; we just subtract it back off at the end.

Our example by the numbers. The colour-choice graph G' of the path has 7 forbidding edges, so we build 7 chains: 6 logical atoms $+\,2 \times 7 = 14$ helper atoms $=$ **20** atoms, and the offset is $C = 2 \times 7 =$ **14**. The best independent set of G' has size 3 (our colouring), so the machine's heaviest set weighs $3 + 14 =$ **17**.

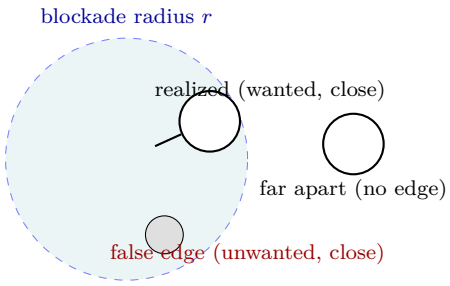
Decoding is trivial. From the heaviest set, **throw away the helper atoms and keep the logical ones**. The logical atoms that survive are exactly the switches we turned on — our independent set, i.e. our colouring.

(Code: `UDGColor.Gadget.gadgetize` and `decodeLogical`; the identity $MWIS(A) = \alpha(G') + C$ is checked against the oracle on all small graphs.)

4 Stage 3: the layout — and the seven heuristics

Up to now A is just an abstract graph: "these atoms are joined, those are not." But the machine has no wires. Two atoms are joined *only if they are physically close* — within a fixed **blockade radius** r. So we must **place every atom of A at a point in the plane** so that:

- every pair the gadget *wants* joined ends up *within* r (a **realized** edge), and
- every pair it wants *apart* ends up *farther* than r; if two atoms are accidentally too close, that is a **false edge**, and if a wanted pair is too far, that is a **missing edge**.



In plain words. This is the *only* job of the heuristics: **geometry**. Because Stage 2 already fixed the right answer exactly, a clumsy layout can only make the problem *not fit* or *harder to solve* — it can **never** produce a wrong colour. That is the whole reason we were allowed to demote the heuristics to "helpers" instead of trusting them with correctness.

The seven heuristics, one by one. They cooperate to find good atom positions p and a good radius r. Here is what each one actually does, in plain terms (and roughly the order they run).

H4 — smart starting positions (runs first). Don't start atoms randomly. Use the graph's own "shape" (its *Laplacian eigenvectors*, a standard way to spread a graph out) to drop every atom roughly where it belongs: joined atoms start near each other, unrelated atoms start far apart. A good start saves the next step a lot of work. *(Code: `fiedlerInit`.)*

H1 — nudge until it fits (the main loop). Treat wanted pairs like little springs pulling close and unwanted-but-too-close pairs like magnets pushing apart, plus a gentle floor that keeps no two atoms on top of each other. Then *nudge* every atom a tiny step downhill, repeatedly, lowering a single "badness" score. We take a fixed number of steps (not "until it magically converges"), and each step is checked so the score truly goes down. *(Code: `descend`; in our example the score falls steadily over the allotted steps.)*

H2 — spend effort where it hurts. Some wanted connections have to snake through many other gadgets and are the expensive, error-prone ones. Weight those more heavily in the badness score, so the nudging focuses on tidying the long, tangled routes rather than the easy short ones.

H3 — don't let atoms pile up. A clump of atoms all within r of each other creates a storm of false edges and wastes hardware. Detect over-crowded neighbourhoods and push the offenders apart. The physics even gives a hard ceiling on how many atoms can legally sit inside one blockade disk, which we respect.

H5 — local touch-ups that must improve. After the big nudging, a few stubborn bad pairs may remain. Fix them one at a time by sliding the two atoms along the line between them — but only *keep* a move if it strictly lowers the *count of broken pairs*. Since that count is a whole number that can only go down, this is guaranteed to stop; it cannot loop forever. *(Code: `repair`.)*

H6 — pick the radius. With the atoms placed, choose the smallest radius $r^\star$ that still captures *every* wanted edge: $r^\star$ = the longest wanted-pair distance. Any smaller and we would lose a needed edge; we report this $r^\star$ and how many false edges it brings in. *(Code: `minFeasibleRadius`.)*

H7 — judge, don't optimise, the discrete stuff. Counts like "number of false edges" jump in steps and have no smooth slope, so you cannot nudge them directly. Instead we use them only to *score and compare* finished layouts (e.g. at the H6 radius choice), never as the thing H1 nudges. *(Code:* `falseEdges`, `missingEdges`.*)*

	role	one-line summary
H4	start	place atoms using the graph's natural shape
H1	drive	nudge atoms downhill on a smooth badness score
H2	weight	put the effort into the long, tangled connections
H3	spread	break up over-crowded clumps of atoms
H5	repair	local moves that must lower the broken-pair count
H6	radius	smallest r that keeps every wanted edge
H7	judge	use discrete quality only to compare layouts

Our example, laid out. For the path's gadget the layout actually succeeds *perfectly*: all wanted edges realized, **zero false edges, zero missing**, at radius $r^\star = 1.3$. (Bigger, denser instances leave some false edges behind — that residue is the honest signal that a truly clean placement needs the specialised "crossing gadgets" we have not built yet. The heuristics report the residue; they never hide it.)

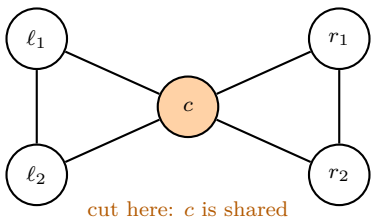
In plain words. Remember: even if the layout were ugly and full of false edges, the *colour we eventually read out is still correct or the layout is rejected as infeasible* — never silently wrong. Geometry decides "does it fit," not "what is the answer."

5 Decomposition: how to colour a graph too big for the machine

The catch: the gadget's atom count grows like $(nk)^2$, so a few-hundred-atom machine runs out of room after only a handful of vertices. The fix uses a simple fact: **colouring is local.** Every rule is a single edge, and edges live inside small neighbourhoods. So cut the graph into small *overlapping pieces*, colour them *one at a time*, and glue the results. The machine only ever holds one small piece.

Cut at a separator. A *separator* is a small set of vertices whose removal splits the graph in two. The friendliest kind is a *clique separator* (the shared vertices are all mutually adjacent), because then gluing is free — explained below.

Example: the bowtie. Two triangles sharing a single vertex c. That shared vertex is a separator of size one (a cut vertex).



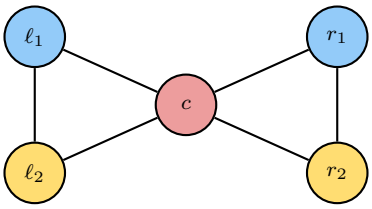
Step 1 — solve the left triangle on its own. Run the whole Stage 1–3 pipeline on just $\{\ell_1, \ell_2, c\}$ with $k = 3$. The machine returns, say,

$$c = 1, \quad \ell_1 = 2, \quad \ell_2 = 3.$$

The device only ever saw a 3-vertex piece, not the whole graph.

Step 2 — solve the right triangle, with c pinned. Now c already has colour 1. We *precolour* it: in the colour-choice graph of the right triangle, vertex c keeps *only* its switch $(c, 1)$ and loses the others. Solve; the machine fills in the rest, e.g. $r_1 = 2$, $r_2 = 3$.

Step 3 — glue. Both pieces now agree that c is colour 1, so their colourings simply merge:



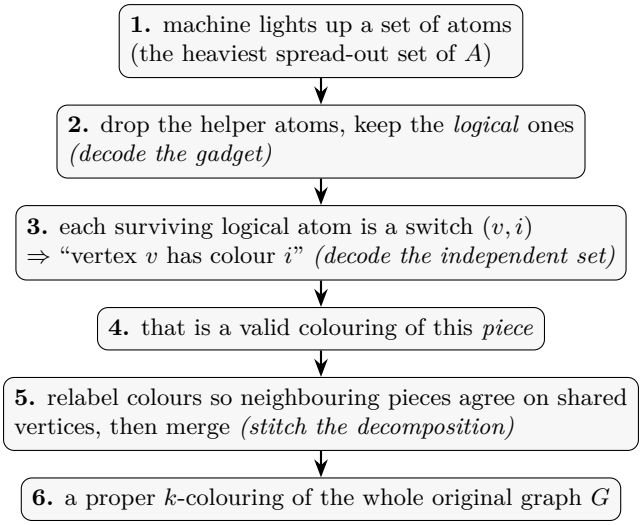
In plain words. Why clique separators glue for free. On the shared clique, every two vertices are adjacent, so any valid colouring gives them *all different* colours. Two pieces might have used different *names* for those colours, but a colouring is only fixed up to renaming — so we just relabel one piece's colours to match the other on the shared vertices. No searching, no guessing.

When the shared vertices are *not* all adjacent. Then renaming isn't enough: we have to *try* the possible colourings of the (small) separator — at most $k^{(\text{separator size})}$ of them — and keep one that works on both sides. Small separators keep this cheap; clique separators make it vanish. Either way, **the machine never holds more than one piece**, so its size depends on the biggest piece, not on how many vertices the whole graph has. A chain of 1000 triangles is coloured by 1000 tiny three-vertex solves.

(In the code, the precolouring already exists as `colorChoiceGraphPrecolored`*; the piece-cutting and stitching driver is the next layer to build.)*

6 Recovering the colours: running every arrow backwards

Finally, the part that ties it all together — turning the machine's raw output back into colours on the *original* graph. It is just every step undone, in reverse:



And one bookkeeping detail: any vertices we peeled off at the start because they had fewer than k neighbours (they can always be coloured last) get their colours filled in greedily at the very end — there is always a free colour left for them.

In plain words. Each arrow we built going *down* (Stages 1, 2, 3, and the cut) has a matching arrow going *up* that recovers the answer. The exact stages (1 and 2) recover it perfectly; the layout (3) is just thrown away once the machine has measured; and the decomposition glue (the cut) is undone by a cheap colour-relabelling. That round trip — a problem compiled down and an answer certified back up — is the whole point of the project.

One-page recap

1. **Colour → independent set.** Make a switch (v, i) for each vertex/colour; forbid two colours on one vertex (choice) and the same colour on neighbours (conflict). A colouring = an independent set of size n.
2. **Independent set → gadget.** Replace each forbidding edge by a weight-1/weight-2 chain of four. The heaviest spread-out set is the best independent set plus a fixed constant C. Decode by keeping the logical atoms.
3. **Gadget → layout.** Place the atoms in the plane so close pairs are exactly the wanted edges. Seven heuristics (start, drive, weight, spread, repair, radius, judge) handle the geometry — and geometry alone.
4. **Big graphs → pieces.** Cut at small separators, solve pieces one at a time with shared vertices pinned, glue by relabelling colours.
5. **Read it back.** Lit atoms → logical atoms → switches → piece colourings → stitched colouring of G.